

---

**Algorithm 0:** Model-level execution flow on the tile array

---

**Require:** Weights  $W_Q, W_K, W_V, W_O$  and the token stream in LPDDR; weights stay resident in 3D DRAM once loaded. Tile groups and their source/owner roles as in Algorithms 1–2.

- 1 Load  $W_Q, W_K, W_V, W_O$  from LPDDR into per-tile 3D DRAM by host DMA. // step 1: weight  $\rightarrow$  3D DRAM
- 2 Bind the static schedule: assign  $\text{src}_Q(y), \text{src}_{KV}(x), \text{dst}_O(y)$  for every tile group. // step 2: schedule  $\rightarrow$  static flow
- 3 Compute  $X = \text{embed}(\text{tokens})$ .
- 4 **for**  $1 \leq l \leq N$  (layers) **do**
- 5     Each tile group: compute  $Q = XW_Q, K = XW_K, V = XW_V$ , write the slices to per-tile 3D DRAM.
- 6     Append  $K, V$  to the KV cache in tile 3D DRAM, evict cold blocks, reload on demand. // step 4: KV  $\rightleftharpoons$  host 3D DRAM
- 7     Run Algorithm 2 on each tile group, compute  $O$ . // step 2 executes data+narrow
- 8     Scheduler marks each group done. // narrow
- 9     Compute  $X = \text{FFN/residual over } O$ , pass it to the next layer. // step 3: feature pipeline data
- 10 **end**
- 11 Write the last-layer  $X$  to host 3D DRAM for the scheduler to sample the next token. // results  $\rightarrow$  host 3D DRAM data
- 12 Return  $\text{tokens} = \text{detokenize}(X)$ .

---

**Placement invariant.** Source tile  $\text{src}_Q(y)$  holds  $Q_{iy}$ , source tile  $\text{src}_{KV}(x)$  holds  $K_{jx}^\top, V_{jx}$ , and output owner  $\text{dst}_O(y)$  collects and stores  $O_{iy}$ . 3D DRAM is tile-local, so a source-tile load is a local read and generates no NoC traffic. Choosing the sources as edge tiles ( $x=0, y=0$ ) or diagonal tiles is a scheduling policy, not a semantic requirement. Cold slices are staged from LPDDR and evicted KV blocks from host 3D DRAM into the owning tile’s 3D DRAM by the model-level placement pipeline, outside this block-level algorithm.

---

**Algorithm 1:** Per-tile fetch (unicast baseline) on a  $G_x \times G_y$  tile group

---

**Require:**  $Q, K^\top, V, O \in \mathbb{R}^{S \times D}$  resident in per-tile 3D DRAM, placed from LPDDR by host DMA before this block. Block sizes  $B_c, B_r$ , tile group sizes  $G_x, G_y$ . In the following algorithm,  $x, y$  denote tile coordinates relative to the group.

- 1 Divide  $Q, O$  into  $T_r = \lceil S/B_r \rceil$  blocks  $Q_1, \dots, Q_{T_r} \in \mathbb{R}^{B_r \times D}$  and  $O_1, \dots, O_{T_r}$ . Divide  $K^\top, V$  into  $T_c = \lceil S/B_c \rceil$  blocks.
- 2 Further divide each  $Q_i, O_i$  into  $G_y$  slices  $Q_{i1}, \dots, Q_{iG_y} \in \mathbb{R}^{(B_r/G_y) \times D}$ , and each  $K_j^\top, V_j$  into  $G_x$  slices. Slices are accessed according to tile coordinates  $x, y$ .
- 3 **for**  $1 \leq i \leq T_r$  **do**
- 4     On chip, initialize  $O_{iy}^{(0x)} = 0$ .
- 5     each tile  $(x, y)$ : read  $Q_{iy}$  from  $\text{src}_Q(y)$ 's 3D DRAM to on-chip L1.     //  $G_x$  unicast reads per slice data
- 6     **for**  $1 \leq j \leq T_c$  **do**
- 7         each tile  $(x, y)$ : read  $K_{jx}^\top, V_{jx}$  from  $\text{src}_{KV}(x)$ 's 3D DRAM to on-chip L1.     //  $G_y$  unicast reads per slice data
- 8         Tile on-chip compute  $S_{iy}^{(jx)} = Q_{iy} K_{jx}^\top$ .
- 9         Tile on-chip compute  $S_{iy}^{(jx)} = S_{iy}^{(jx)} / \sqrt{D}$ ,  $m_{iy}^{(jx)} = \text{rowmax}(S_{iy}^{(jx)})$ .
- 10        **if**  $j > 1$  **then**
- 11          Tile on-chip compute  $m_{iy}^{(jx)} = \max(m_{iy}^{(j-1)}, m_{iy}^{(jx)})$ .
- 12        **end**
- 13        (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $m_{iy}^{(j)} = \max(m_{iy}^{(jx)})$  in tile group.     // narrow
- 14        (row owner  $\text{dst}_O(y)$ ) Row-wise multicast  $m_{iy}^{(j)}$  in tile group.     // narrow
- 15        Tile on-chip compute  $\tilde{P}_{iy}^{(jx)} = \exp(S_{iy}^{(jx)} - m_{iy}^{(j)})$ .
- 16        Tile on-chip compute  $\ell_{iy}^{(jx)} = \text{rowsum}(\tilde{P}_{iy}^{(jx)})$ .
- 17        (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $\ell_{iy}^{(j)} = \sum(\ell_{iy}^{(jx)})$  in tile group.     // narrow
- 18        (row owner  $\text{dst}_O(y)$ ) Row-wise multicast  $\ell_{iy}^{(j)}$  in tile group.     // narrow
- 19        **if**  $j > 1$  **then**
- 20          Tile on-chip compute  $\ell_{iy}^{(j)} = e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}} \ell_{iy}^{(j-1)} + \ell_{iy}^{(j)}$ .
- 21          Tile on-chip compute  $O_{iy}^{(jx)} = \text{diag}(e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}}) O_{iy}^{(j-1)x}$ .
- 22        **end**
- 23        Tile on-chip compute  $O_{iy}^{(jx)} = O_{iy}^{(jx)} + \tilde{P}_{iy}^{(jx)} V_{jx}$ .
- 24        Tile on-chip update  $m_{iy}^{(j-1)} \leftarrow m_{iy}^{(j)}$ ,  $\ell_{iy}^{(j-1)} \leftarrow \ell_{iy}^{(j)}$ .
- 25     **end**
- 26     Tile on-chip compute  $O_{iy}^{(T_c)x} = \text{diag}(\ell_{iy}^{(T_c)})^{-1} O_{iy}^{(T_c)x}$ .
- 27     (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $O_{iy}^{(T_c)} = \sum(O_{iy}^{(T_c)x})$  in tile group.     // data
- 28     (row owner  $\text{dst}_O(y)$ ) Write  $O_{iy}$  to its local 3D DRAM as the  $y$ -th slice in  $i$ -th block of  $O$ .     // local write
- 29 **end**

---

---

**Algorithm 2:** Multicast on a  $G_x \times G_y$  tile group (FlatAttention structure on per-tile 3D DRAM)

---

**Require:**  $Q, K^\top, V, O \in \mathbb{R}^{S \times D}$  resident in per-tile 3D DRAM, placed from LPDDR by host DMA before this block. Block sizes  $B_c, B_r$ , tile group sizes  $G_x, G_y$ . In the following algorithm,  $x, y$  denote tile coordinates relative to the group.

- 1 Divide  $Q, O$  into  $T_r = \lceil S/B_r \rceil$  blocks  $Q_1, \dots, Q_{T_r} \in \mathbb{R}^{B_r \times D}$  and  $O_1, \dots, O_{T_r}$ . Divide  $K^\top, V$  into  $T_c = \lceil S/B_c \rceil$  blocks.
- 2 Further divide each  $Q_i, O_i$  into  $G_y$  slices  $Q_{i1}, \dots, Q_{iG_y} \in \mathbb{R}^{(B_r/G_y) \times D}$ , and each  $K_j^\top, V_j$  into  $G_x$  slices. Slices are accessed according to tile coordinates  $x, y$ .
- 3 **for**  $1 \leq i \leq T_r$  **do**
- 4     On chip, initialize  $O_{iy}^{(0x)} = 0$ .
- 5     (row source  $\text{src}_Q(y)$ ) Load  $Q_{iy}$  from local 3D DRAM to tile on-chip L1.     // local read
- 6     (row source  $\text{src}_Q(y)$ ) Row-wise multicast  $Q_{iy}$  in tile group.     // data
- 7     **for**  $1 \leq j \leq T_c$  **do**
- 8         (column source  $\text{src}_{KV}(x)$ ) Load  $K_{jx}^\top, V_{jx}$  from local 3D DRAM to tile on-chip L1.     // local read
- 9         (column source  $\text{src}_{KV}(x)$ ) Column-wise multicast  $K_{jx}^\top, V_{jx}$  in tile group.     // data
- 10         Tile on-chip compute  $S_{iy}^{(jx)} = Q_{iy} K_{jx}^\top$ .
- 11         Tile on-chip compute  $S_{iy}^{(jx)} = S_{iy}^{(jx)} / \sqrt{D}$ ,  $m_{iy}^{(jx)} = \text{rowmax}(S_{iy}^{(jx)})$ .
- 12         **if**  $j > 1$  **then**
- 13             Tile on-chip compute  $m_{iy}^{(jx)} = \max(m_{iy}^{(j-1)}, m_{iy}^{(jx)})$ .
- 14         **end**
- 15         (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $m_{iy}^{(j)} = \max(m_{iy}^{(jx)})$  in tile group.     // narrow
- 16         (row owner  $\text{dst}_O(y)$ ) Row-wise multicast  $m_{iy}^{(j)}$  in tile group.     // narrow
- 17         Tile on-chip compute  $\tilde{P}_{iy}^{(jx)} = \exp(S_{iy}^{(jx)} - m_{iy}^{(j)})$ .
- 18         Tile on-chip compute  $\ell_{iy}^{(jx)} = \text{rowsum}(\tilde{P}_{iy}^{(jx)})$ .
- 19         (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $\ell_{iy}^{(j)} = \sum(\ell_{iy}^{(jx)})$  in tile group.     // narrow
- 20         (row owner  $\text{dst}_O(y)$ ) Row-wise multicast  $\ell_{iy}^{(j)}$  in tile group.     // narrow
- 21         **if**  $j > 1$  **then**
- 22             Tile on-chip compute  $\ell_{iy}^{(j)} = e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}} \ell_{iy}^{(j-1)} + \ell_{iy}^{(j)}$ .
- 23             Tile on-chip compute  $O_{iy}^{(jx)} = \text{diag}(e^{m_{iy}^{(j-1)} - m_{iy}^{(j)}}) O_{iy}^{(j-1)x}$ .
- 24         **end**
- 25         Tile on-chip compute  $O_{iy}^{(jx)} = O_{iy}^{(jx)} + \tilde{P}_{iy}^{(jx)} V_{jx}$ .
- 26         Tile on-chip update  $m_{iy}^{(j-1)} \leftarrow m_{iy}^{(j)}$ ,  $\ell_{iy}^{(j-1)} \leftarrow \ell_{iy}^{(j)}$ .
- 27     **end**
- 28     Tile on-chip compute  $O_{iy}^{(T_c)x} = \text{diag}(\ell_{iy}^{(T_c)})^{-1} O_{iy}^{(T_c)x}$ .
- 29     (row owner  $\text{dst}_O(y)$ ) Row-wise reduce  $O_{iy}^{(T_c)} = \sum(O_{iy}^{(T_c)x})$  in tile group.     // data
- 30     (row owner  $\text{dst}_O(y)$ ) Write  $O_{iy}$  to its local 3D DRAM as the  $y$ -th slice in  $i$ -th block of  $O$ .     // local write
- 31 **end**

---